

Functional Features Included – ComicCraft

ComicCraft – AI Comic Story Creator using Gemini Models provides an AI-assisted workflow for converting a user's simple story idea into structured comic content containing characters, scenes, dialogues and illustrations. The functional features below are aligned with the project's problem statement, technology stack and solution architecture.

1. Story Idea Input

Purpose: Provides the starting point for comic creation.

User Input: The user enters a basic story idea through the web interface.

System Processing: The frontend collects the story idea and sends it to the Flask backend for processing.

Output: A story prompt/request ready for AI-assisted generation.

2. AI Story Generation

Purpose: Converts the user's initial idea into structured story content.

User Input: Story idea and relevant details supplied through the interface.

System Processing: The Flask backend prepares the request and communicates with the Gemini API/models.

Output: AI-generated story content that can be organized into comic elements.

3. Character Generation

Purpose: Defines characters required by the generated story.

User Input: Character requirements may be derived from the user's story idea.

System Processing: AI processing identifies and generates suitable character information from the story context.

Output: Character descriptions/details that support the comic narrative.

4. Scene Generation

Purpose: Breaks the generated story into meaningful comic scenes.

User Input: Generated story content.

System Processing: The AI-assisted workflow organizes story events into scene-based content.

Output: A sequence of scenes suitable for comic presentation.

5. Dialogue Generation

Purpose: Provides dialogue for characters within the generated scenes.

User Input: Characters and scene context.

System Processing: Gemini-based generation produces dialogue related to the story and scene context.

Output: Character dialogues associated with the comic scenes.

6. Comic Illustration Generation

Purpose: Adds visual illustrations to the generated comic content.

User Input: Generated scene and comic requirements.

System Processing: AI-based image generation is used to produce visual content for the comic.

Output: Illustrations associated with the generated comic scenes.

7. Structured Comic Output

Purpose: Combines the generated elements into a coherent comic flow.

User Input: Generated characters, scenes, dialogues and illustrations.

System Processing: The backend prepares the generated content into an organized result.

Output: A structured comic story containing textual and visual elements.

8. User-Friendly Web Interface

Purpose: Provides a simple interaction point for users.

User Input: Story idea and required information.

System Processing: HTML, CSS and JavaScript handle the user-facing interface and interaction.

Output: A web-based interface through which users can submit input and view generated results.

9. Backend Processing

Purpose: Coordinates application workflow between the interface and AI services.

User Input: Request received from the frontend.

System Processing: Python and Flask receive the request, manage the application flow and communicate with AI services.

Output: Processed AI response and comic-generation result for the frontend.

10. Generated Result Display

Purpose: Presents the completed comic result to the user.

User Input: Generated story and illustrations returned by the backend.

System Processing: The frontend receives and presents the generated content.

Output: User-visible comic story and visual output.

11. End-to-End Functional Flow

1. User opens the ComicCraft web interface.
2. User enters a story idea.
3. The frontend sends the request to the Flask backend.
4. Flask manages the request and prepares the AI interaction.
5. Gemini models process the story prompt.
6. Story content is organized into characters, scenes and dialogues.
7. AI-based image generation produces comic illustrations.
8. The backend prepares the generated textual and visual content.
9. The frontend displays the generated comic result to the user.

12. Expected Functional Inputs and Outputs

Input	Processing	Output
Story idea	Gemini-assisted story processing	Structured story content
Story context	Character and scene generation	Characters and scenes
Scene context	Dialogue generation	Character dialogues
Generated scene requirements	AI image generation	Comic illustrations
Generated comic elements	Backend organization	Structured comic result

13. Feature-to-Technology Mapping

Feature	Technology / Component
Web interface and user interaction	HTML, CSS, JavaScript
Application workflow	Python, Flask
AI story and comic content generation	Google Gemini API / Gemini Models
Comic visual generation	AI-based image generation
Runtime configuration	Environment Variables (.env)
Source management	Git & GitHub

14. Functional Requirements Checklist

- User can provide a story idea through the web interface.
- The request is handled by the Flask backend.
- Gemini-based processing is used for AI-assisted story generation.

- Characters can be generated from the story context.
- Scenes can be organized from the generated story.
- Dialogues can be generated for characters and scenes.
- AI-based image generation supports comic illustrations.
- Generated comic elements are organized into a structured output.
- The generated result is presented through the web interface.

15. Functional Outcome

The combined functional features provide a complete AI-assisted comic creation workflow. ComicCraft accepts a story idea, processes it through the Flask and Gemini-based architecture, generates characters, scenes, dialogues and illustrations, and presents the resulting comic content to the user through the web interface.